Mechanistic Proofs of AI Training Stability: A Formal Inquiry

Executive Summary

This report addresses a critical epistemic gap in artificial intelligence: the disparity between the empirical success of techniques for stabilizing model training and the lack of foundational, mechanistic proofs explaining *why* they work. While interventions such as specific normalization layers, regularization methods, and initialization strategies are widely observed to improve convergence and robustness, the underlying causal pathways remain subjects of debate. To bridge this gap, this document functions as a comprehensive inquiry into the principles of AI training stability, moving beyond correlation to establish a framework for verifiable, causal explanation.

The analysis begins by deconstructing the ambiguous notion of "stability" into a multi-dimensional, formal lexicon. It defines core instability phenomena like Catastrophic Forgetting and Mode Collapse and proposes a framework for stability based on measurable properties of convergence, fidelity, coherence, and gradient normality. To quantify these dimensions, the report introduces novel and existing metrics, including a **Semantic Drift Score (SDS)** based on the Population Stability Index (PSI) applied to latent space and a **Confidence-Fidelity Divergence (CFD)** metric.

Subsequently, the report provides a mechanistic inventory of common stability interventions. It categorizes techniques across regularization, normalization, and initialization, hypothesizing their causal impact on the loss landscape, gradient flow, and feature distributions. This analysis reveals that these interventions form a tightly coupled system of controls, where choices in one area (e.g., normalization) have profound, often non-obvious implications for others (e.g., effective learning rate and regularization).

To validate these hypotheses, a latent space observability framework is proposed. This framework leverages Topological Data Analysis (TDA) to create a

Chrono-Topological Signature (CTS) of a model's internal representations, allowing for the visualization of structural changes over time. Complementing this, neuro-symbolic probes, built with differentiable logic layers, are proposed for verifying specific logical properties within the latent space.

Building on this observational capacity, the report outlines paradigms for formal verification. Recognizing the scalability limitations of traditional methods, it focuses on the CROWN family of verifiers (CROWN, β-CROWN, SDP-CROWN), which combine the efficiency of GPU-accelerated bound propagation with the rigor of complete verification methods like branch-and-bound. For properties that are semantic in nature, the Neuro-Symbolic Assertion Language (NESAL) is presented as a powerful tool for creating verifiable specifications.

Finally, a reflexive audit of these proposed frameworks acknowledges their limitations, primarily the persistent trade-off between verifier scalability and tightness, and the inherent difficulty of specification. This critique leads to the generation of novel, high-priority research trajectories, formatted as a library of declarative prompts for future inquiry. The report concludes that while a universal theory of stability remains distant, the synthesis of advanced observability (TDA), scalable formal methods (CROWN), and expressive specification languages (Neuro-Symbolic AI) provides a viable and rigorous pathway toward designing and deploying provably stable and resilient AI systems.

Section 1: A Formal Lexicon of AI Training Stability

1.1 Introduction: The Ambiguity of Stability

The discourse surrounding the training of advanced AI models is replete with terms like "stability," "robustness," and "resilience." While intuitively understood, these terms lack the formal precision necessary for rigorous scientific inquiry and engineering. This ambiguity creates an "epistemic gap": we can observe that an intervention, such as adding a Layer Normalization step, improves training, but a consensus on the precise mechanism is often elusive. To move from empirical art to verifiable science, a

shared, formal lexicon is required. This section deconstructs "stability" into its constituent phenomena, proposes a multi-dimensional definition, and introduces quantifiable metrics to ground the concept in measurable reality.[1]

1.2 Core Instability Phenomena: Definitions and Manifestations

Training instability manifests in several well-documented failure modes. Understanding these phenomena is the first step toward a comprehensive definition of stability.

1.2.1 Catastrophic Forgetting (CF)

- **Definition:** First formally introduced by McCloskey and Cohen, Catastrophic Forgetting (CF) describes the tendency of a neural network, when trained sequentially on new tasks, to abruptly and drastically degrade its performance on previously learned tasks.[2] It is a central challenge in the field of Continual Learning (CL), which aims to build systems that can learn incrementally without forgetting.[5]
- **Causal Factors:** The fundamental cause of CF is the shift in data distribution as the model moves from one task to the next. The optimization process, focused on minimizing loss on the current task's data, overwrites weights that were critical for performance on past tasks.[3] In Deep Reinforcement Learning (DRL), this is exacerbated by the "deadly triad" of function approximation, bootstrapping, and off-policy training, which leads to non-stationary targets and data distributions, making interference between old and new knowledge more pronounced.[2]
- **Manifestation:** CF is a failure of *purpose fidelity* over time. A model proficient at Task A, after being fine-tuned on Task B, may exhibit near-random performance when re-evaluated on Task A.

1.2.2 Mode Collapse

- **Definition:** Mode collapse is a distinct failure mode, most prominently observed

in Generative Adversarial Networks (GANs), where the generator fails to capture the full diversity of the true data distribution, instead producing a limited, repetitive set of outputs.[6] This represents a failure of *representational completeness*.

- **Causal Factors:** One hypothesis posits that due to non-uniform sampling during training, the GAN objective function can reach its minimum even when the generated distribution does not match the real one, creating local optima where diversity is not rewarded.[6] Another view links mode collapse to instabilities in the discriminator's learning process, where it fails to provide a useful gradient signal, sometimes leading to a "spectral collapse" in the singular values of its weight matrices.[8] In Mixture-of-Experts (MoE) models, a similar phenomenon occurs when the gating mechanism routes most inputs to a few "winning" experts, causing other experts to become underutilized or inactive, thereby collapsing the model's effective capacity.[9]
- **Manifestation:** A GAN trained on a diverse image dataset like CIFAR-10 might only generate images of cars, ignoring all other classes. An MoE model may show that only a small fraction of its experts receive any inputs during inference.

1.2.3 Semantic Drift and Algorithmic Trauma

- **Definition:** Expanding on these classical phenomena, this report introduces two higher-level concepts from the DRP. **Semantic Drift** is the gradual or sudden change in a model's internal representation of a concept, leading to a divergence between the model's output and the intended meaning. **Algorithmic Trauma**, or "semantic scars," refers to persistent, pathological structures in the latent space that result from severe training instability events, such as a major episode of CF or exposure to adversarial data.[10]
- **Connection to Core Phenomena:** CF is a specific, measurable instance of semantic drift where the meaning of past tasks is lost. Mode collapse is a form of semantic drift where the model's representation of an entire distribution collapses onto a few points. The phenomenon of "massive activations" in Pre-LN Transformers, where large activation spikes persist across layers, could be considered a type of semantic scar, biasing the model's focus.[11]

1.3 A Multi-Dimensional Framework for Stability

Based on these phenomena, AI Training Stability can be formally defined not as a single property, but as a vector of interdependent qualities that must be maintained throughout the training process. A system is stable if it exhibits:

1. **Convergence:** The training process reaches a state of low loss on the objective function in a finite and predictable manner, without wild oscillations or divergence.
2. **Purpose Fidelity:** The model retains its capabilities on previously learned tasks or data distributions when exposed to new ones, demonstrating resistance to harmful catastrophic forgetting.
3. **Representational Coherence:** The model's latent space maintains a well-structured and separable representation of the data distribution, avoiding pathologies like mode collapse, expert underutilization, or semantic fragmentation.
4. **Gradient Normality:** The norms of the gradients remain within a bounded, non-zero range throughout training, avoiding the vanishing or exploding gradient problems that halt learning.
5. **Performance Consistency:** The model's final performance on a held-out validation set exhibits low variance across multiple training runs initiated with different random seeds, indicating a smooth and reliable optimization landscape.

A critical nuance is the duality of forgetting. While CF is typically harmful, some forms of forgetting are beneficial. Forgetting can mitigate overfitting to training data, discard irrelevant or noisy information to facilitate learning new knowledge, or be intentionally induced for privacy-preserving machine unlearning.[3] This reframes stability not as perfect memory (zero plasticity), but as

selective plasticity: the ability to update knowledge in a controlled manner that improves performance on a target distribution without unacceptably degrading it on other valued distributions. This transforms stability from a monolithic goal into a context-dependent optimization problem.

1.4 Quantifying Stability: Proposed Metrics

To make the multi-dimensional framework operational, we must define quantifiable

metrics.

- **Population Stability Index (PSI):** PSI is a well-established statistical metric used to measure the shift between two probability distributions.[13] It is calculated by binning the data and applying the formula:PSI=i=1Σn(Actuali%−Expectedi%)×ln(Expectedi%Actuali%)

  where 'Expected' refers to a baseline distribution and 'Actual' refers to a current distribution. A PSI below 0.1 indicates no significant shift, while a value above 0.2 suggests a significant shift requiring investigation.15
- **Proposed Metric 1: Semantic Drift Score (SDS):** The abstract concept of "Semantic Drift" can be made concrete by applying a statistical drift metric to the model's semantic representations. The SDS is defined as the PSI calculated between the distribution of latent space embeddings of a fixed set of concept inputs at two different points in the training process, t1 and t2.
  - SDS = PSI(Embeddings(concepts, t1), Embeddings(concepts, t2))
    This directly quantifies how much the model's internal "understanding" of these concepts has shifted, providing a powerful tool for detecting representational instability.16
- **Proposed Metric 2: Confidence-Fidelity Divergence (CFD):** This metric quantifies the alignment between a model's predictive confidence (e.g., the maximum softmax probability) and its actual accuracy (fidelity).[10]
  - CFD = E[confidence] - Accuracy on a held-out validation set.
    A large positive CFD indicates overconfidence, a common symptom of overfitting or poor calibration, which are forms of instability.
- **Proposed Metric 3: Latent Topological Complexity (LTC):** Drawing from Topological Data Analysis (TDA), this metric quantifies the "shape" of the latent space. It can be defined as a summary of the persistence diagram (e.g., total persistence or number of significant Betti numbers) computed from the activation vectors of a given layer.[18] Research suggests that stable training and good generalization are correlated with a "topological simplification" process, where the LTC of the latent space decreases as the model learns to separate data classes into simpler manifolds.[18]

These metrics, summarized in Table 1, provide the necessary tools to move the study of stability from a qualitative to a quantitative domain.

Table 1: Structured Lexicon of AI Training Stability

Term	Formal Definition	Observable Manifestation	Hypothesized Causal Factors	Proposed Metrics	Key References
Catastrophic Forgetting	The tendency of a network to abruptly forget previously learned knowledge upon learning new, sequentially presented information.	A significant drop in performance on Task A after fine-tuning on Task B.	Shift in data distribution between tasks; weight overwriting during optimization.	Task-specific accuracy decay.	[2]
Mode Collapse	A failure of a generative model to capture the full diversity of the true data distribution, producing limited and repetitive outputs.	A GAN trained on diverse digits generates only '1's; Inactive experts in an MoE model.	Non-uniform sampling; unstable discriminator dynamics; insufficient generator capacity.	Inception Score, Fréchet Inception Distance (FID), Expert Utilization Rate.	[6]
Semantic Drift	A gradual or sudden change in a model's internal representation of a concept, leading to a divergence from intended meaning.	A model mislabels source entities as target entities after few-shot fine-tuning.[20]	Catastrophic forgetting; fine-tuning on out-of-distribution data.	**Semantic Drift Score (SDS)**	[17]
Confidence-Fidelity	The misalignmen	A model is 99%	Overfitting; poor model	**CFD** = E[confidenc	[10]

Divergence (CFD)	t between a model's predictive confidence and its true accuracy (fidelity).	confident on predictions where it is only 70% accurate.	calibration; training on noisy labels.	e] - Accuracy.	
Latent Topological Complexity (LTC)	A measure of the structural complexity (e.g., number of holes/voids) of the latent space activation manifold.	High Betti numbers in the persistence diagram of latent activations.	Poorly separated class representati ons; noisy or entangled features.	**LTC** (e.g., total persistence).	[18]

Section 2: A Mechanistic Inventory of Stability Interventions

To bridge the epistemic gap, it is essential to move from knowing *that* an intervention works to hypothesizing *how* it works at a computational and mathematical level. This section provides a systematic inventory of common stability interventions, categorizing them and detailing their hypothesized mechanistic pathways.

2.1 Regularization Techniques: Constraining Complexity

Regularization techniques are designed to prevent overfitting and improve generalization, which are intimately linked to training stability.[22] They achieve this by adding constraints or penalties to the learning process.

2.1.1 Weight-based Regularization (L2, Weight Decay, CPR)

- **Description:** These methods directly penalize the magnitude of model parameters. The most common forms are L2 regularization and its decoupled variant, weight decay, which apply a uniform penalty to all parameters proportional to the square of their values.[23] A more advanced technique, Constrained Parameter Regularization (CPR), enforces an adaptive upper bound on a statistical measure (e.g., the L2-norm) of individual parameter matrices, using an augmented Lagrangian method to solve the constrained optimization problem.[23]
- **Mechanistic Hypothesis:** The primary mechanism is the **smoothing of the loss landscape**. By penalizing large weights, these techniques discourage the model from fitting the training data with overly complex functions that create sharp, narrow minima in the loss landscape. Such minima are inherently unstable, as small shifts in the input data can lead to large changes in output. The adaptive nature of CPR suggests a more targeted smoothing effect, applying regularization only where needed and thus avoiding unnecessary restrictions on the model's capacity.[23]

2.1.2 Stochastic Regularization (Dropout, Shakeout, Noise Injection)

- **Description:** These methods introduce randomness into the training process. Dropout, the most well-known, temporarily and randomly sets a fraction of neuron activations to zero during each training iteration.[22] Shakeout is an extension that not only drops units but also implicitly induces L0 and L1 regularization, encouraging sparser weights.[24] Other methods involve adding noise directly to gradients or parameters.[22]
- **Mechanistic Hypothesis:** The core mechanism is the **prevention of complex co-adaptations** between neurons. By randomly disabling neurons, Dropout forces the network to learn more robust and redundant features, as it cannot rely on any single neuron being present. This can be viewed as training a large ensemble of thinned networks and averaging their predictions, which smooths the optimization trajectory. Shakeout's sparsity-inducing effect further simplifies the learned model by mechanistically pruning less important connections, thereby reducing complexity and improving stability.[24]

2.1.3 Uncertainty-based Regularization (PUN, BBB)

- **Description:** In contrast to modeling weights as single point estimates, techniques like Parameter Uncertainty Networks (PUN) and Bayes-by-Backprop (BBB) treat each weight as a probability distribution.[25] The loss function is augmented with a Kullback-Leibler (KL) divergence term, which penalizes the learned weight distributions for deviating from a predefined prior distribution.
- **Mechanistic Hypothesis:** This approach provides regularization by explicitly **constraining the volume of the function space** the model can represent. The KL-divergence term ensures that the posterior distributions of the weights stay close to the prior, preventing the model from fitting the training data too precisely. This is particularly effective for improving stability in non-stationary environments, such as Reinforcement Learning, where it helps prevent overfitting to transient aspects of the environment's dynamics.[25]

2.2 Normalization Layers: Taming Internal Distributions

Normalization layers are a cornerstone of modern deep learning architectures, widely credited with improving training stability and speed.[26]

2.2.1 The Core Mechanism: From Covariate Shift to Landscape Smoothing

- **Description:** Normalization layers like Batch Normalization (BN) and Layer Normalization (LN) re-center and re-scale the activations within a network on a per-batch or per-layer basis.[27] The canonical explanation for their effectiveness is the mitigation of "internal covariate shift"—the idea that changes in the distribution of a layer's inputs during training complicate learning for subsequent layers.[29]
- **Mechanistic Hypothesis (Deeper View):** While mitigating distributional shifts is a factor, a more fundamental mechanism appears to be the **smoothing of the optimization landscape**.[27] By ensuring activations remain in a well-behaved range, normalization allows for the use of much higher learning rates without causing divergence. Furthermore, a subtle but powerful mechanism has been

identified: normalization makes the preceding subnetwork scale-invariant. This means that as the norm of the weights in that subnetwork grows during training (a typical occurrence), the
effective learning rate for that subnetwork implicitly shrinks. This implicit, adaptive learning rate decay is a powerful, often overlooked, stabilizing mechanism that helps prevent loss of plasticity.[30]

2.2.2 The Placement Dilemma: Pre-LN vs. Post-LN vs. Peri-LN

In Transformer architectures, the placement of Layer Normalization relative to the residual connection has profound stability implications.

- **Post-LN (Norm(x + Module(x))):** This is the original Transformer design.[31] The mechanism normalizes the main information pathway after the residual addition. While it can lead to better final model performance, it is notoriously difficult to train. The normalization on the main path can weaken gradient signals, leading to **gradient vanishing** in deep models and necessitating careful learning rate warm-up to prevent initial divergence.[11]
- **Pre-LN (x + Module(Norm(x))):** This has become the de facto standard for large language models due to its superior training stability.[32] The mechanism leaves the residual connection unnormalized, creating a "clean" gradient path back through the network. This prevents gradient vanishing but introduces a new problem: **exponential growth in activation variance**. As signals accumulate along the unnormalized residual path, their variance can grow layer by layer, leading to "massive activations" that dominate the sub-layer outputs and reduce the learning capacity of deeper layers.[11]
- **Peri-LN:** An emerging hybrid approach that applies normalization both before and after the sub-layer (i.e., on both its input and output).[11] The mechanism aims to strike a balance, moderating the variance growth seen in Pre-LN while avoiding the gradient vanishing of Post-LN. It provides a more controlled evolution of hidden states, offering a promising path toward stable large-scale training.[31]

2.2.3 Other Innovations (HybridNorm, QK-Norm, NaP)

- **HybridNorm:** Strategically combines different normalization schemes, for

instance, using QKV normalization within the attention mechanism and Post-Norm in the feed-forward networks, to leverage the strengths of each.[29]

- **QK-Norm:** Applies normalization directly to the query (Q) and key (K) components before the attention score computation, which has been shown to enhance stability.[29]
- **Normalize-and-Project (NaP):** This technique explicitly decouples the implicit learning rate decay from normalization. It achieves this by coupling normalization layers with a weight projection step, ensuring the effective learning rate remains constant. This makes the learning rate schedule explicit and improves robustness in non-stationary settings.[30]

2.3 Initialization Strategies: Setting the Stage for Stability

The state of a network at the beginning of training is a critical determinant of stability. Initialization methods aim to set initial parameter values that facilitate effective signal propagation.[36]

2.3.1 The Goal: Signal Propagation and Dynamical Isometry

- **Description:** Early methods like Xavier initialization were designed to maintain the variance of signals as they propagate forward and backward, preventing them from vanishing or exploding in the non-saturated region of activation functions.[38] A stricter and more powerful condition is **dynamical isometry**, which seeks to preserve the entire spectrum of singular values of the input-output Jacobian, ensuring that information propagates without being stretched or squashed in any direction.[38]
- **Mechanistic Hypothesis:** By ensuring that signal (and gradient) norms are preserved across layers at initialization, these methods place the network in a "trainable" region of the parameter space from the outset, allowing optimization to proceed smoothly.

2.3.2 Identity-Control Initialization (Fixup, IDInit)

- **Description:** These methods are specifically designed for residual networks and aim to make each residual block behave like an identity function at initialization. This reduces the need for normalization layers. The Fixup method achieves this by, among other things, setting the weights of the last layer in each residual branch to zero.[38]
- **IDInit Mechanism:** The "fully identical initialization" (IDInit) method improves upon this by initializing *all* layers in the residual branch to be identity-like. It uses a padded identity-like matrix to handle non-square weight matrices and cleverly overcomes the associated convergence challenges by using Stochastic Gradient Descent (SGD) with momentum, which introduces asymmetry into the gradients and breaks the pathological symmetry of an identity-initialized system.[38] The mechanism creates a perfectly clean "information highway" through the network at initialization, ensuring stable signal propagation even in very deep architectures without normalization.

The analysis of these interventions reveals that they are not independent components but a system of interacting controls. For example, IDInit is proposed as an alternative to normalization layers, showing a direct trade-off. The NaP method reveals that normalization creates an implicit learning rate schedule that interacts with the explicit regularization of weight norms. Therefore, a mechanistic proof for any single intervention is incomplete if it does not account for its interaction with the broader system of controls governing the network's dynamics.

Table 2: Stability Intervention Matrix

Intervention Category	Specific Technique	Hypothesized Mechanistic Pathway	Impact on Key Dynamics	Known Trade-offs/Constraints	Key References
Regularization	L2 / Weight Decay	Smooths loss landscape by penalizing large weights, discouraging sharp minima.	Loss Landscape, Gradient Flow	Uniform penalty may be overly restrictive for some parameters.	23

	Dropout	Prevents neuron co-adaptation by creating an ensemble of thinned networks.	Feature Distribution	Can slow down initial convergence; introduces randomness.	22
	Shakeout	Induces L0/L1 sparsity in addition to Dropout's effect, simplifying the model.	Feature Distribution	More complex than standard Dropout.	24
Normalization	Post-LN	Normalizes the main residual path, providing strong regularization.	Gradient Flow	Prone to gradient vanishing, requires careful warm-up.	11
	Pre-LN	Stabilizes initial gradients by keeping the residual path unnormalized.	Gradient Flow, Feature Distribution	Allows exponential growth of activation variance ("massive activations").	11
	Peri-LN	Normalizes both input and output of sub-layers to balance variance and gradients.	Gradient Flow, Feature Distribution	Aims for a balance, but mechanisms are still under investigation.	11
	NaP	Decouples normalization from implicit learning rate	Gradient Flow	Makes learning rate schedule explicit, improving	30

		decay via weight projection.		robustness to non-stationarity.	
Initialization	Xavier / Kaiming	Maintains signal variance during forward/backward passes to prevent vanishing/exploding gradients.	Gradient Flow	Assumes linear activations or specific non-linearities (ReLU).	38
	IDInit	Initializes residual blocks as identity functions, creating a clean information path.	Gradient Flow, Feature Distribution	Reduces or eliminates the need for normalization layers.	38

Section 3: Probing Latent Space Dynamics for Mechanistic Insight

To move from hypothesis to proof, we require methods to observe and verify the internal mechanics of stability. This section outlines a conceptual framework for such observation, focusing on techniques that can probe the high-dimensional latent spaces where semantic representations are formed and where the effects of stability interventions manifest.

3.1 An Observability Protocol for Latent Space

A robust observability protocol must capture the internal state of a model throughout its training lifecycle. This involves designing a pipeline to:

1. **Instrument the Model:** Intercept and record activation vectors from key layers at regular training intervals (e.g., every 100 steps).
2. **Generate Probing Data:** Pass a fixed, representative set of input data through the model at each interval to generate the activations for analysis. This ensures that observed changes are due to the model's evolution, not changes in the input distribution.
3. **Compute Signatures:** Apply the analytical techniques described below (TDA, neuro-symbolic probes) to the recorded activations to generate a time-series of quantitative and qualitative signatures.
4. **Create an Auditable Trace:** Log these signatures alongside standard metrics (loss, accuracy) to create a comprehensive, auditable record linking external performance to internal dynamics.

3.2 Topological Data Analysis (TDA): Mapping the "Shape of Meaning"

Topological Data Analysis (TDA) offers a powerful lens for understanding the high-dimensional geometry of latent spaces, moving beyond simple statistics to characterize their intrinsic structure or "shape".[18]

3.2.1 Core TDA Tools: Persistent Homology and Mapper

- **Persistent Homology:** This is the central tool of TDA. It examines a dataset by building a sequence of simplicial complexes (a generalization of graphs to higher dimensions) at varying spatial resolutions. It tracks the "birth" and "death" of topological features—such as connected components (H0), loops (H1), and voids (H2)—as the resolution changes. The results are summarized in a **persistence diagram**, a multiset of points where each point's coordinates represent the birth and death scale of a feature. The distance of a point from the diagonal indicates the feature's persistence or significance.[18]
- **Mapper:** This algorithm provides a simplified, graph-based visualization of high-dimensional data. It clusters points based on their proximity and the value of a "filter function" (e.g., centrality or density), then represents these clusters as

nodes in a graph, with edges connecting nodes that share common points. The resulting graph reveals the large-scale connectivity and structure of the data.[18]

3.2.2 Application: The Chrono-Topological Signature (CTS)

The **Chrono-Topological Signature (CTS)** is a novel concept proposed here to operationalize the use of TDA for stability analysis. It is defined as a time-series of topological summaries (e.g., persistence diagrams or their vector representations like persistence landscapes) computed from a model's latent space activations at regular training intervals.

This CTS provides a dynamic "fingerprint" of the model's learning process. It allows for testing mechanistic hypotheses directly. For instance, the hypothesis that stable training involves "topological simplification" can be verified by observing whether the number of significant topological features (e.g., Betti numbers, which count the features of each dimension) in the CTS consistently decreases as the model learns to separate concepts into distinct, simpler manifolds.[18] By comparing the CTS of a model trained with a stabilizing intervention against a baseline, one can visualize and quantify the intervention's impact on the evolution of latent space structure.

3.2.3 Simulating and Observing "Algorithmic Trauma"

The CTS is particularly well-suited for studying the effects of "algorithmic trauma" as specified in the DRP. A simulation protocol could involve:

1. Training a model on Task A and recording its final CTS, representing a stable state.
2. Training the model on a new, conflicting Task B, inducing catastrophic forgetting.
3. Retraining the model on Task A and recording the new final CTS.
   By comparing the initial and recovered CTS, one can quantify the "semantic scar." Does the latent topology for Task A return to its original state, or is it permanently altered? Stabilizing interventions, like the pseudo-rehearsal methods described in 2, could then be applied during step 2, and their mitigating effect on this topological scarring could be directly observed in the CTS.

3.3 Neuro-Symbolic Probes: Verifying Latent Properties

While TDA is excellent for discovery, neuro-symbolic techniques offer a path toward targeted verification of specific, logical properties within the latent space.

3.3.1 Differentiable Logic Layers as Verifiable Probes

- **Description:** Differentiable Logic Gate Networks (DLGNs) are architectures composed of layers of logic operators (e.g., AND, OR, XOR).[40] By using continuous, differentiable relaxations of these discrete logic functions (e.g., representing AND as multiplication
  a * b for inputs a, b ∈), the entire network, including the choice of which logic gate to use at each node, can be trained end-to-end with gradient descent.[42]
- **Application as Probes:** We can design a "propositional probe" by attaching a small, trainable DLGN to an intermediate layer of a larger model under study. This probe can be trained to verify a specific logical property of the latent representations. For example, to test if the representations of two classes, A and B, are linearly separable, the probe could be trained to implement the proposition IS_SEPARABLE(embedding_A, embedding_B). The probe's loss function then becomes a direct, differentiable measure of how well the main network's latent space satisfies this desired logical structure.

A crucial aspect of this approach is the "observer effect." The act of attaching and training such a probe is not passive. The gradients from the probe's loss function will backpropagate into the main network, effectively acting as a form of regularization that actively encourages the network to form representations that satisfy the property being probed. This transforms the probe from a mere measurement device into an active tool for enforcing desired latent space geometries.

3.3.2 Neuro-Symbolic AI for Constraint Enforcement

Beyond simple probes, the broader paradigm of Neuro-Symbolic AI, which integrates neural perception with symbolic reasoning, offers a powerful framework for ensuring stability.[44] A symbolic reasoner, equipped with a knowledge base (e.g., a knowledge graph), can act as a "Symbolic Continuity Guardian".[21] For example, if a language model is being fine-tuned, this guardian could monitor the embeddings of key concepts. If the embedding for "Paris" (the city) drifts semantically closer to the embedding for "Paris Hilton" (the person), the knowledge graph would identify this as a violation of the

is_a(city) property, flagging the semantic drift in real-time.

These two approaches—exploratory TDA and targeted neuro-symbolic verification—are not mutually exclusive. They form a powerful discovery-verification loop. TDA can be used in an exploratory phase to analyze the CTS of many training runs, generating hypotheses about which topological features correlate with stability. Once a hypothesis is formed (e.g., "stable models have fewer 1-dimensional holes in their latent manifold"), a neuro-symbolic probe can be designed to explicitly penalize the model for creating such features. Success in this verification phase provides strong causal evidence for the importance of the TDA-discovered property.

Section 4: Paradigms for the Formal Verification of Stability Mechanisms

Building upon the observational frameworks, this section outlines methodologies for constructing formal, mathematical proofs of the stability mechanisms hypothesized in Section 2. The goal is to establish causality with a level of rigor that transcends empirical correlation.

4.1 Formalizing the Verification Problem

The standard neural network verification problem can be stated as follows: given a network function $f_\theta(x)$, a set of inputs X (often an L_p-norm ball around a point), and a set of outputs Y, prove that for all inputs $x \in X$, the output $f_\theta(x)$ is in Y.[47] This can be

expressed as:

$$\forall x \in X \Rightarrow f_\theta(x) \in Y$$

Many stability properties can be cast into this form. For instance, proving robustness to catastrophic forgetting for Task A after fine-tuning on Task B can be formulated as: let f' be the fine-tuned model, X be the set of valid inputs for Task A, and Y be the set of correct outputs for Task A. The verification task is to prove that $\forall x \in X \Rightarrow f'(x) \in Y$.

4.2 Complete Verification via Branch-and-Bound with Optimized Bound Propagation

While verification methods based on Satisfiability Modulo Theories (SMT) or Mixed-Integer Linear Programming (MILP) can provide exact proofs, they do not scale to the size of modern neural networks.[48] The most promising direction for scalable verification lies in methods that combine efficient but incomplete bound propagation with complete search algorithms.

- **Bound Propagation (CROWN):** The CROWN (Convex Relaxation-based O(n^2) Worst-case-scenario) algorithm is a highly efficient but incomplete verification method.[49] It works by recursively propagating linear bounds for each neuron's activation backwards through the network. This process is highly parallelizable and can be executed efficiently on GPUs, but the relaxation required can be too loose to prove difficult properties.[50]
- **Complete Verification (β-CROWN):** The β-CROWN algorithm builds upon CROWN to create a *complete* verifier that is orders of magnitude faster than traditional LP-based methods.[50] It employs a **Branch-and-Bound (BaB)** search, where it recursively splits the input domain or neuron activation states (e.g., for a ReLU neuron, considering the cases where its input is positive or negative). The key innovation is its ability to encode these split constraints directly into the bound propagation process via a set of optimizable parameters, denoted as β. By optimizing these β parameters using gradient ascent, β-CROWN can compute bounds that are as tight as those from expensive LP solvers but with the efficiency of bound propagation.[50]
- **State-of-the-Art Enhancements (GCP-CROWN, SDP-CROWN):** The CROWN framework is a living field of research. Recent enhancements like SDP-CROWN incorporate principles from tighter but slower Semidefinite Programming (SDP) relaxations into the scalable bound propagation pipeline, improving tightness by

better capturing inter-neuron dependencies.[51] GCP-CROWN further tightens bounds by integrating cutting-plane methods, which add valid linear constraints to prune the search space.[53]

4.3 Neuro-Symbolic Verification: Proving Complex Semantic Properties

For many crucial stability properties, the primary challenge is not the verification algorithm itself, but the act of specification. How does one formally define "semantic coherence" in a machine-checkable way? The **Neuro-Symbolic Assertion Language (NESAL)** provides a powerful solution.[55]

The core idea of NESAL is to allow a logical assertion to include calls to other neural networks, known as "specification networks." These networks act as proxies for complex, semantic concepts. For example, to prove that a self-driving car's perception model f does not confuse pedestrians with traffic cones, one could use a highly accurate, pre-verified specification network g that detects pedestrians. The property to verify, expressed in NESAL, would be: IF g(input) = "pedestrian" THEN f(input) ≠ "traffic_cone". This transforms an intractable semantic specification into a tractable verification problem over the inputs and outputs of two networks, which can then be solved by composing their respective bounds.

4.4 A Blueprint for Proving a Mechanistic Hypothesis

Let us apply these paradigms to a concrete hypothesis from Section 2.

- **Hypothesis:** The Pre-LN normalization strategy contributes to stability by controlling the magnitude of gradients during training, preventing them from exploding.
- Formalization: The property to verify is that for a network f with Pre-LN, the L2-norm of the gradient of the loss L with respect to the parameters θ is bounded by some constant C for all inputs in the training distribution D.
  $\forall x \in D \Rightarrow \|\nabla_\theta L(f_\theta(x))\|_2 \leq C$
- **Proof Methodology:**
  1. **Construct the Gradient Graph:** The computation of the gradient via backpropagation is itself a sequence of linear and non-linear operations. One

can construct the explicit computational graph that takes network inputs x and parameters θ and outputs the gradient vector ∇θL.

2. **Apply a Verifier:** Apply a complete, bound-propagation-based verifier like β-CROWN to this gradient computational graph.
3. **Compute Bounds:** The verifier will compute provable lower and upper bounds for each individual component of the gradient vector.
4. **Derive Norm Bound:** From these component-wise bounds, a provable upper bound on the L2-norm of the entire gradient vector can be derived.
5. **Verify Property:** If this proven upper bound is less than or equal to the chosen constant C, the property is formally verified for the given network and input domain. This constitutes a rigorous, mechanistic proof of the hypothesis.

It is crucial to recognize the spectrum of proof strength. A TDA analysis provides compelling *empirical evidence*. A result from an incomplete verifier like CROWN provides a *partial proof*. A successful run of a complete verifier like β-CROWN provides a *deterministic proof* for a specific model instance. A mathematical derivation that holds for an entire class of architectures represents the strongest form of proof. Epistemic humility requires being precise about which level of proof is being claimed.

Furthermore, verification should not be seen merely as a post-hoc analysis. Because bound propagation methods like CROWN are differentiable, the certified robustness bound itself can be incorporated as a penalty term in the training loss. This approach, known as **certified training**, actively steers the optimization process toward regions of the parameter space that are not only accurate but also provably robust, representing a paradigm shift toward "correctness-by-construction".[56]

Table 3: Mechanistic Proof Paradigm Blueprint

Stability Mechanism to Prove	Formal Property to Verify (Example)	Proposed Verification Paradigm	Key Tools/Techniques	Assumptions & Limitations	Key References
Pre-LN Controls Gradient Growth	$\forall x \in D,		\nabla_{\theta} L(f_{\theta}(x))		_2 \le C$

Dropout Prevents Co-adaptation	The removal of any single neuron k does not change the output class: f(x)=fk-removed(x).	Complete verification comparing the original network to a pruned version.	β-CROWN, Model Checking	Computationally expensive as it requires one verification run per neuron.	22
IDInit Enables Norm-Free Training	A deep ResNet with IDInit and no normalization layers converges to loss Lfinal<ϵ.	This is an empirical claim, not a formal property. Proof is via demonstration.	Empirical training runs and loss curve analysis.	Does not provide a formal guarantee, but strong empirical evidence.	38
Model Resists Semantic Drift	IF g(x) = "cat" THEN f'(x) ≠ "dog" where f′ is fine-tuned and g is a spec network.	Neuro-Symbolic Verification.	NESAL, α,β-CROWN	Relies on the correctness and robustness of the specification network g.	55

Section 5: Reflexive Audit and Future Research Trajectories

This final section conducts a critical evaluation of the proposed frameworks, acknowledging their limitations and inherent assumptions. It then synthesizes the findings to chart a course for future inquiry, embodying the role of a "Self-Evolving Planner" to guide the next generation of research into AI training stability.

5.1 Reflexive Critique: Assumptions and Limitations

The path toward provably stable AI is paved with significant challenges that demand epistemic humility.

- **The Scalability vs. Tightness Trade-off:** This is the most fundamental challenge in formal verification. Efficient, bound-propagation-based methods like β-CROWN are scalable to large networks but may produce bounds that are too loose to prove a desired property (a "false negative").[50] Conversely, tighter methods based on SDP or MILP can prove more properties but are computationally intractable for all but the smallest models.[52] The proofs outlined in this report are therefore contingent on the verifier's ability to terminate with a conclusive answer within a reasonable time budget.
- **The Specification Problem:** Formal verification is only as strong as its specification. While NESAL offers a powerful way to express complex semantic properties, it outsources the difficulty to the "specification network".[55] The proof is then conditional on the correctness and robustness of this specification network, pushing the problem one level deeper rather than solving it entirely. For many desirable qualities, like "fairness" or "creativity," crafting a complete and unambiguous formal specification remains an open research problem.
- **The Generalization Gap in Proofs:** Many theoretical analyses and formal proofs are conducted on simplified or smaller-scale models (e.g., fully-connected networks with ReLU activations). Generalizing these proofs to the vast and complex architectures used in practice, such as Mixture-of-Experts Transformers with diverse activation functions and intricate residual pathways, is a non-trivial leap. A proof for a simplified model provides valuable insight but is not a guarantee for a production-scale system.

5.2 Generalizability Assessment

The mechanistic principles identified in this report likely exhibit a degree of generalizability, but their specific implementations do not. For example, the underlying principle that **controlling activation variance is crucial for stable training in deep networks** is likely to hold true across CNNs, RNNs, and Transformers. However, the specific mechanism and its parameters will differ. The scaling factor in DeepNorm for Transformers was derived specifically for that architecture's self-attention and FFN blocks.[32] Applying it to a ResNet would be meaningless; a new analysis would be required to derive an equivalent scaling factor suitable for convolutional blocks. Therefore, while the

what (the principle) is generalizable, the *how* (the mechanism) is

architecture-specific.

This analysis reveals that stability is not the result of a single component but is an **emergent property of a complex system**. The interactions between architecture, optimizer, data, initialization, normalization, and regularization create a dynamic system with intricate feedback loops. The search for a single "magic bullet" intervention is therefore likely futile. The future of designing stable AI lies in the co-design and co-adaptation of all these components, viewing them not as a list of independent choices but as a unified control system.

5.3 Novel Research Questions: A Generative Prompt Library

The gaps and insights identified throughout this report give rise to a set of high-priority, testable research questions. These are framed as declarative prompts to guide future automated or human-led research.

The research process itself—Observe -> Hypothesize -> Formalize -> Prove -> Audit—is an epistemic loop. The ultimate frontier in this field is the automation of this loop, creating a self-organizing system that accelerates our collective understanding of AI. TDA can automate hypothesis generation (discovery), verifiers can automate hypothesis testing (proof), and a "Self-Evolving Planner" can automate the auditing and re-planning stages. The following prompts represent seeds for such an advanced research ecosystem.

Table 4: Generative Prompt Design Pattern Library

Prompt ID	Prompt Type	Declarative Prompt	Core Concepts Involved	Key References
SP-001	BiasAudit	Given the finding that "beneficial forgetting" can mitigate overfitting, design a BiasAudit protocol. The	Beneficial Forgetting, Fairness, Formal Verification, Algorithmic Trauma	[3]

		protocol must use formal verification (e.g., β-CROWN) to prove that the forgetting induced by regularization technique R does not disproportionately degrade performance on protected subgroups from the original training data, thereby preventing the introduction of fairness-related "algorithmic trauma."		
SP-002	PolicyImplementation	Design a PolicyImplementation for a novel normalization layer, 'Adaptive-LN'. This layer must dynamically interpolate its behavior between Pre-LN and Post-LN based on a real-time measurement of latent space instability. Use a Chrono-Topological Signature (CTS) to quantify instability and formalize the	Pre-LN/Post-LN Dynamics, TDA, Chrono-Topological Signature, Control Theory	[11]

		interpolation policy to be provably bounded within a stable regime.		
SP-003	TheoreticalInquiry	Develop a unified theoretical framework using principles from control theory and dynamical systems to model the coupled dynamics of weight norms, normalization statistics, and explicit learning rate schedules. Use this framework to derive a priori conditions for training stability and design a 'meta-optimizer' that co-adapts all parameters to provably remain within a stable region.	Dynamical Systems, Control Theory, Normalization, Regularization	[23]
SP-004	ProofOfConcept	Construct a formal, mechanistic proof using neuro-symbolic verification (NESAL) that the pseudo-rehearsal technique described in [2]	Catastrophic Forgetting, Neuro-Symbolic Verification, Pseudo-rehearsal	[2]

		mitigates catastrophic forgetting. The specification network must formalize the concept of "knowledge retention" for a past task, and the proof must show that this property holds for the model trained with pseudo-rehearsal but is violated for the naively fine-tuned model.		

5.4 Actionable Recommendations for Practitioners

While the grand challenge of provable stability is a long-term research endeavor, the insights from this report yield immediate, actionable recommendations for AI engineers and researchers building systems today.

1. **For Architecture Design:** Avoid monolithic choices. For large Transformer models, prioritize hybrid or balanced normalization schemes like **Peri-LN** [11] or **HybridNorm** [29] over pure Pre-LN or Post-LN to better manage the trade-off between gradient stability and activation growth. Consider using modern initialization schemes like **IDInit** [38] in residual architectures to reduce the reliance on complex normalization schemes altogether.
2. **For Training and Debugging:** Implement proactive monitoring of internal model states. Use the **Population Stability Index (PSI)** to calculate a **Semantic Drift Score (SDS)** on latent space embeddings for a fixed validation set. This provides an early warning system for representational instability, allowing for intervention before catastrophic performance degradation occurs.[15]
3. **For High-Assurance Systems:** In safety-critical domains (e.g., medical

diagnostics, autonomous vehicles), shift from a post-hoc testing mindset to a **correctness-by-construction** paradigm. Integrate **certified training** techniques into the development lifecycle. By including a differentiable, certified robustness bound directly in the training objective, models can be optimized to be not only performant but also provably robust from the start.[56]

Works cited

1. Review of Metrics to Measure the Stability, Robustness and Resilience of Reinforcement Learning - ResearchGate, accessed July 16, 2025, https://www.researchgate.net/publication/359435810_Review_of_Metrics_to_Measure_the_Stability_Robustness_and_Resilience_of_Reinforcement_Learning
2. Pseudo-Rehearsal: Achieving Deep Reinforcement Learning without Catastrophic Forgetting - arXiv, accessed July 16, 2025, http://arxiv.org/pdf/1812.02464
3. A Comprehensive Survey of Forgetting in Deep Learning Beyond Continual Learning - arXiv, accessed July 16, 2025, https://arxiv.org/html/2307.09218v3
4. A Comprehensive Survey of Forgetting in Deep Learning ... - arXiv, accessed July 16, 2025, https://arxiv.org/pdf/2307.09218
5. Advancements and Challenges in Continual Reinforcement Learning: A Comprehensive Review - arXiv, accessed July 16, 2025, https://arxiv.org/html/2506.21899
6. Distribution Fitting for Combating Mode Collapse in Generative Adversarial Networks - arXiv, accessed July 16, 2025, https://arxiv.org/html/2212.01521v2
7. HingeRLC-GAN: Combating Mode Collapse with Hinge Loss and RLC Regularization, accessed July 16, 2025, https://arxiv.org/html/2503.19074v1
8. Spectral Regularization for Combating Mode Collapse in GANs - CVF Open Access, accessed July 16, 2025, https://openaccess.thecvf.com/content_ICCV_2019/papers/Liu_Spectral_Regularization_for_Combating_Mode_Collapse_in_GANs_ICCV_2019_paper.pdf
9. Improving Deep Learning Performance with Mixture of Experts and Sparse Activation, accessed July 16, 2025, https://www.preprints.org/manuscript/202503.0611/v1
10. The Epistemic Architect: Cognitive Operating System : r/PromptEngineering - Reddit, accessed July 16, 2025, https://www.reddit.com/r/PromptEngineering/comments/1m0u7qy/the_epistemic_architect_cognitive_operating_system/
11. Peri-LN: Revisiting Layer Normalization in the Transformer Architecture - arXiv, accessed July 16, 2025, https://arxiv.org/html/2502.02732v1
12. Peri-LN: Revisiting Normalization Layer in the Transformer Architecture - arXiv, accessed July 16, 2025, https://arxiv.org/html/2502.02732v3
13. Stability index: A tool for assessing ML models - DataScientest, accessed July 16, 2025, https://datascientest.com/en/stability-index-a-tool-for-assessing-ml-models
14. Population Stability Index (PSI): What You Need To Know - Arize AI, accessed July

16, 2025, https://arize.com/blog-course/population-stability-index-psi/
15. Population Stability Index | Matthew Burke's Blog, accessed July 16, 2025, https://mwburke.github.io/data%20science/2018/04/29/population-stability-index.html
16. Comparing multiple latent space embeddings using topological analysis - ResearchGate, accessed July 16, 2025, https://www.researchgate.net/publication/363052338_Comparing_multiple_latent_space_embeddings_using_topological_analysis
17. (PDF) Contextually Entangled Gradient Mapping for Optimized LLM Comprehension, accessed July 16, 2025, https://www.researchgate.net/publication/388657619_Contextually_Entangled_Gradient_Mapping_for_Optimized_LLM_Comprehension
18. Topological Data Analysis for Neural Network Analysis: A ... - UB, accessed July 16, 2025, https://www.ub.edu/topologia/casacuberta/articles/TDASurvey.pdf
19. Topological Data Analysis for Neural Network Analysis: A Comprehensive Survey - arXiv, accessed July 16, 2025, https://arxiv.org/html/2312.05840v2
20. Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers) - ACL Anthology, accessed July 16, 2025, https://aclanthology.org/volumes/2024.naacl-long/
21. Natural Language Processing - Reddit, accessed July 16, 2025, https://www.reddit.com/r/LanguageTechnology/rising/
22. Generalization in neural networks: a broad survey - arXiv, accessed July 16, 2025, https://arxiv.org/html/2209.01610v3
23. Improving Deep Learning Optimization through Constrained Parameter Regularization - arXiv, accessed July 16, 2025, https://arxiv.org/html/2311.09058v3
24. Shakeout: A New Approach to Regularized Deep Neural Network Training - arXiv, accessed July 16, 2025, https://arxiv.org/pdf/1904.06593
25. Regularized Parameter Uncertainty for Improving Generalization in Reinforcement Learning - CVF Open Access, accessed July 16, 2025, https://openaccess.thecvf.com/content/CVPR2024/papers/Moure_Regularized_Parameter_Uncertainty_for_Improving_Generalization_in_Reinforcement_Learning_CVPR_2024_paper.pdf
26. [2505.11312] Where You Place the Norm Matters: From Prejudiced to Neutral Initializations - arXiv, accessed July 16, 2025, https://arxiv.org/abs/2505.11312
27. Understanding and Improving Layer Normalization - arXiv, accessed July 16, 2025, https://arxiv.org/pdf/1911.07013
28. [1607.06450] Layer Normalization - arXiv, accessed July 16, 2025, https://arxiv.org/abs/1607.06450
29. HybridNorm: Towards Stable and Efficient Transformer Training via Hybrid Normalization - arXiv, accessed July 16, 2025, https://arxiv.org/html/2503.04598v1
30. Normalization and effective learning rates in reinforcement learning - arXiv, accessed July 16, 2025, https://arxiv.org/html/2407.01800v1
31. Peri-LN: Revisiting Layer Normalization in the Transformer Architecture - arXiv, accessed July 16, 2025, https://arxiv.org/pdf/2502.02732?

32. GPAS: Accelerating Convergence of LLM Pretraining via Gradient-Preserving Activation Scaling - arXiv, accessed July 16, 2025, https://arxiv.org/html/2506.22049v1
33. What is Reinforcement Learning? - ResearchGate, accessed July 16, 2025, https://www.researchgate.net/post/What_is_Reinforcement_Learning
34. Peri-LN: Revisiting Layer Normalization in the Transformer Architecture - Hugging Face, accessed July 16, 2025, https://huggingface.co/papers/2502.02732
35. [2502.02732] Peri-LN: Revisiting Normalization Layer in the Transformer Architecture - arXiv, accessed July 16, 2025, https://arxiv.org/abs/2502.02732
36. Starting Positions Matter: A Study on Better Weight Initialization for Neural Network Quantization - arXiv, accessed July 16, 2025, https://arxiv.org/html/2506.10463v1
37. Advancing Neural Network Performance through Emergence ... - arXiv, accessed July 16, 2025, https://arxiv.org/html/2407.19044
38. IDInit: A Universal and Stable Initialization Method for Neural ..., accessed July 16, 2025, https://arxiv.org/pdf/2503.04626
39. IDInit: A Universal and Stable Initialization Method for Neural Network Training - arXiv, accessed July 16, 2025, https://arxiv.org/html/2503.04626v2
40. Deep Differentiable Logic Gate Networks - NIPS, accessed July 16, 2025, https://papers.neurips.cc/paper_files/paper/2022/file/0d3496dd0cec77a999c98d35003203ca-Paper-Conference.pdf
41. Learning Interpretable Differentiable Logic Networks - arXiv, accessed July 16, 2025, https://arxiv.org/html/2407.04168v1
42. Differentiable Logic Cellular Automata, accessed July 16, 2025, https://google-research.github.io/self-organising-systems/difflogic-ca/
43. Deep Differentiable Logic Gate Networks - OpenReview, accessed July 16, 2025, https://openreview.net/forum?id=vF3WefcoePW
44. Neuro-Symbolic AI - Codefinity, accessed July 16, 2025, https://codefinity.com/blog/Neuro-Symbolic-AI
45. Q&A: Can Neuro-Symbolic AI Solve AI's Weaknesses? - TDWI, accessed July 16, 2025, https://tdwi.org/articles/2024/04/08/adv-all-can-neuro-symbolic-ai-solve-ai-weaknesses.aspx
46. r/LanguageTechnology - Reddit, accessed July 16, 2025, https://www.reddit.com/r/LanguageTechnology/
47. Formal Verification of Neural Networks for Safety-Critical Tasks in Deep Reinforcement Learning, accessed July 16, 2025, https://proceedings.mlr.press/v161/corsi21a/corsi21a.pdf
48. Simplifying Neural Networks Using Formal Verification - Stanford CS Theory, accessed July 16, 2025, https://theory.stanford.edu/~barrett/pubs/GFM+20.pdf
49. GCP CROWN Slides - Neural Networks Verification with General Cutting Planes, accessed July 16, 2025, https://neurips.cc/media/neurips-2022/Slides/53350.pdf
50. Beta-CROWN: Efficient Bound Propagation with Per-neuron Split ..., accessed July 16, 2025, https://www.cs.columbia.edu/~suman/docs/beta_crown_efficient_bound_pro.pdf

51. SDP-CROWN: Efficient Bound Propagation for Neural Network Verification with Tightness of Semidefinite Programming | OpenReview, accessed July 16, 2025, https://openreview.net/forum?id=5liHhkgvAn
52. SDP-CROWN: Efficient Bound Propagation for Neural Network Verification with Tightness of Semidefinite Programming - arXiv, accessed July 16, 2025, https://arxiv.org/html/2506.06665v1
53. Scalable Neural Network Verification with Branch-and-bound Inferred Cutting Planes - arXiv, accessed July 16, 2025, https://arxiv.org/abs/2501.00200
54. Scalable Neural Network Verification with Branch-and-bound Inferred Cutting Planes - arXiv, accessed July 16, 2025, https://arxiv.org/html/2501.00200v1
55. Neuro-Symbolic Verification of Deep Neural Networks - MPG.PuRe, accessed July 16, 2025, https://pure.mpg.de/rest/items/item_3388679_3/component/file_3388680/content
56. Formal Verification of Deep Neural Networks: Theory and Practice ..., accessed July 16, 2025, https://neural-network-verification.com/